

MarketRadar — README

Setup, run steps, tests, demo data and configuration · October 2026

"Turn live market signals into smarter business decisions."

MarketRadar is an industry-agnostic market-intelligence platform for new and growing businesses. Given a **market**, a **location** and optionally a **product**, it discovers the competing businesses, checks what they offer and at what verified price, tracks how those signals change over time, and summarises the result as evidence-referenced facts.

It is built around one rule: **report what was observed and how strongly it is evidenced — never invent a conclusion**. "Price unavailable" is not "product unavailable", "not in the latest search" is not "closed", and one reading seen twice is not a trend.

Document	PDF	Purpose
Project Document	PDF	Full project document: problem, design, architecture, modules, data, API, testing, limitations
Demo Video Guide	PDF	Step-by-step script for the submission video, and which test data to use
Pre-Submission Audit	PDF	Audit findings, submission blockers and the fix log
Module Working & Execution Flow	PDF	Per-module execution flow with sequence diagrams
Technical Documentation	PDF	Earlier long-form technical reference (the audit fix log records later changes)
This README	PDF	Setup and run steps

1. Modules

Module	Page	Endpoint(s)	Spends SerpApi credits?
Market Research	/research	GET /api/serpapi/search , /api/serpapi/recent	Yes, 1 per uncached search
Competitors	/competitors	GET /api/competitors/search , /api/competitors	Yes, 1 per uncached search
Products	/products	GET /api/products/search , /api/products	Yes, 1 + up to 5 (one per merchant)
Price Intelligence	/prices	GET /api/prices/local	Yes, about 3 + one per nearby merchant with a website
Trends	/trends	GET /api/trends/*	No — reads stored observations

Module	Page	Endpoint(s)	Spends SerpApi credits?
Market Pulse	/pulse	GET /api/market-pulse, /api/market-pulse/options	No
AI Analyst	/analyst	GET /api/analyst/analysis	No

Search responses are cached for 7 days, so repeating an identical search within a week costs nothing. Location lookup uses SerpApi's free location catalogue and also costs nothing.

2. Technology

- **Frontend:** React 19, TypeScript, Vite 8, Material UI 9, React Router 7, Axios
- **Backend:** Python 3.12, FastAPI, Pydantic 2, SQLAlchemy 2
- **Database:** SQLite (backend/data/market_radar.db , created automatically)
- **Market data:** SerpApi — Google Search (organic + local), Google Maps, Google Shopping
- **AI:** a vendor-neutral AIProvider interface. Facts are computed by code; a model may only narrate them, and its text is validated before display.

3. Prerequisites

Tool	Version	Check with
Python	3.12 (tested on 3.12.3)	python --version
Node.js	20.19+ or 22.12+ (tested on 24.18) — required by Vite 8	node --version
npm	10+ (tested on 11.16)	npm --version
SerpApi key	optional — needed only for the four search modules	https://serpapi.com/manage-api-key

Without a SerpApi key, the app still runs. The search modules then return "SerpApi key not configured" (HTTP 503), and Trends, Market Pulse and the AI Analyst work on whatever history the database holds — or on the demo data from step 6.

4. Run the project — exact steps

Open **two terminals**, one for the backend and one for the frontend. All commands start from the project root (the folder containing this README).

4.1 Backend (terminal 1)

Windows (PowerShell)

```
cd backend
python -m venv venv
.\venv\Scripts\Activate.ps1
pip install -r requirements.txt -r requirements-dev.txt
Copy-Item .env.example .env
notepad .env          # set SERPAPI_API_KEY=<your key>, save, close
uvicorn app.main:app --reload --port 8001
```

If PowerShell blocks `Activate.ps1`, run `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` once, or use `.\venv\Scripts\activate.bat` from cmd.

macOS / Linux

```
cd backend
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt -r requirements-dev.txt
cp .env.example .env
nano .env          # set SERPAPI_API_KEY=<your key>
uvicorn app.main:app --reload --port 8001
```

Check it: open <http://localhost:8001/api/health>. It should show `{"status":"ok","service":"market-radar-api"}`. Interactive API docs are at <http://localhost:8001/docs>.

The first start creates `backend/data/market_radar.db` and its tables. It never drops or overwrites existing data.

4.2 Frontend (terminal 2)

Windows (PowerShell)

```
cd frontend
npm install
Copy-Item .env.example .env
npm run dev
```

macOS / Linux

```
cd frontend
npm install
cp .env.example .env
npm run dev
```

Open <http://localhost:5173>. The sidebar footer shows **Backend Connected** when the API is reachable. Copying `.env` is optional: the frontend defaults to `http://127.0.0.1:8001/api`.

4.3 Every later run

```
# terminal 1
cd backend; .\venv\Scripts\Activate.ps1; uvicorn app.main:app --reload --port 8001
# terminal 2
cd frontend; npm run dev
```

(On macOS/Linux, use `source venv/bin/activate` and `&&` instead of `;`.)

5. Run the tests

```
cd backend
# activate the venv first (see 4.1)
pytest
```

Expected result: **525 passed**. The tests:

- run against a throwaway database (`tests/_test_market_radar.db`) that `tests/conftest.py` creates and deletes. They refuse to run if pointed at the real database.
- never call SerpApi: every provider response is a scripted stub, so the suite spends no credits and needs no API key or network.

Frontend checks:

```
cd frontend
npm run build      # TypeScript type-check (tsc -b) + production build
npm run lint       # oxlint
```

6. Test data

There are three kinds of test data. Use the right one for the job.

6.1 Automated test data (backend unit/integration tests)

This data lives inside the test files (`backend/tests/*.py`) and `tests/stubs.py` . It uses fictional businesses on reserved `.example` domains and scripted SerpApi payloads. `pytest` builds it and deletes it on every run, so there is nothing to set up.

6.2 Demo data (no API key, no credits)

This is synthetic history for showing Trends, Market Pulse and the AI Analyst end to end. It is written to a **separate** database file and never touches `market_radar.db` .

```
cd backend
.\venv\Scripts\Activate.ps1
python scripts\seed_demo_data.py          # creates data\demo_market_radar.db
$env:DATABASE_URL="sqlite:///./data/demo_market_radar.db"
uvicorn app.main:app --reload --port 8001
```

```
# macOS / Linux
python scripts/seed_demo_data.py
DATABASE_URL=sqlite:///./data/demo_market_radar.db uvicorn app.main:app --reload --port 8001
```

It contains two contexts, both at location **"Sample City (demo data)"**:

Context	What it demonstrates
Coffee Shops (two searches, 10 days apart)	Real trend directions: price 14.99 → 15.99; search positions swap; rating 4.5 → 4.6. A business absent from the latest search is flagged "not in latest" (not "closed"). A product found but with no verifiable price. A price read twice unchanged stays "insufficient history". Price statistics from 3 distinct prices.
Bike Repair (one search)	Every series is "insufficient history" — one measurement is never a trend

- Rebuild the demo file with `python scripts/seed_demo_data.py --reset`.
- To return to real data, close the terminal (or run `Remove-Item Env:DATABASE_URL` in PowerShell / `unset DATABASE_URL` in bash) and start the backend normally.

6.3 Live search inputs (uses SerpApi credits)

These inputs produce good results on the live provider:

Page	Market / Product	Location	Notes
Competitors	Wine Retail	Holmdel, NJ	Local + organic results, rejected directories shown
Competitors	Coffee Shops	Austin, Texas	
Products	product Kendall Jackson Vintner's Reserve Chardonnay 750ml, market Wine Retail	Holmdel, NJ	Shows found / verified price / unknown side by side
Price Intelligence	Cycling Frog 5mg Grapefruit 12oz 6pk, radius 25	08807	Nearby-store funnel
Market Research	Wine stores	Holmdel, NJ	

These inputs show validation and cost nothing (rejected before any search):

Input	Result
Competitors with no location	"A location is required..."
Location asdfqwerzx	"could not be matched to a supported search location"

Input	Result
Location <code>HoImdel, Germany</code>	Refused: "'Germany' does not match the closest supported location"
Market <code>x</code>	"too short to describe a market"

7. Configuration

`backend/.env` (template: `backend/.env.example`):

Variable	Required	Default	Purpose
<code>SERPAPI_API_KEY</code>	for search modules	—	SerpApi key
<code>DATABASE_URL</code>	no	<code>sqlite:///./data/market_radar.db</code>	database location (relative to <code>backend/</code>)
<code>CORS_ORIGINS</code>	no	<code>localhost/127.0.0.1 on 5173 and 5174</code>	origins allowed to call the API
<code>AI_PROVIDER</code> , <code>AI_API_KEY</code> , <code>AI_MODEL</code>	no	empty	narrative summaries. They need a registered adapter; none is bundled.
<code>FRONTEND_URL</code>	no	—	informational

`frontend/.env` (template: `frontend/.env.example`):

Variable	Default	Purpose
<code>VITE_API_BASE_URL</code>	<code>http://127.0.0.1:8001/api</code>	backend address

8. Project structure

MarketRadar/	
├─ backend/	
│ └─ app/	
│ └─ main.py	FastAPI app, CORS, table creation
│ └─ config.py	settings from .env
│ └─ routes/	HTTP endpoints (thin: validation → service → status code)
│ └─ services/	business logic (discovery, identity, trends, pulse, analyst)
│ └─ providers/	merchant-website evidence provider
│ └─ models/	SQLAlchemy tables
│ └─ schemas/	Pydantic request/response models
│ └─ database/	engine, session, additive column sync
│ └─ scripts/seed_demo_data.py	
│ └─ tests/	pytest suite (isolated DB, no live calls)
│ └─ data/	SQLite database (git-ignored)
├─ frontend/	
│ └─ src/	
│ └─ pages/	one folder per module
│ └─ services/	API client per module
│ └─ types/	TypeScript response types
│ └─ components/ layouts/ theme/	
├─ docs/	project document, demo guide, audit
└─ _quarantine/	superseded files kept for reference (not part of the app)

9. Troubleshooting

Symptom	Cause / fix
Sidebar shows Backend Unavailable	Backend not running on port 8001, or VITE_API_BASE_URL points elsewhere
Search returns "SerpApi key not configured" (503)	Set SERPAPI_API_KEY in backend/.env and restart the backend
"could not be matched to a supported search location" (422)	Use a city, a postal code, or "City, Full State Name" (e.g. Holmdel, New Jersey)
"The search provider did not respond" (502)	SerpApi error or quota exhausted — check the SerpApi dashboard
Trends say "insufficient history" everywhere	Expected until the same search is repeated after the 7-day cache expires. Use the demo data (6.2) to see directions
Market Pulse / AI Analyst list is empty	No observations yet — run a Competitors or Products search, or use the demo data
npm run dev fails with a Node version error	Upgrade Node to 20.19+ / 22.12+
Port 5173 in use	Vite moves to 5174 automatically; the backend allows both
Activate.ps1 cannot be loaded	Set-ExecutionPolicy -Scope CurrentUser RemoteSigned

10. Known limitations

- **No AI vendor adapter is bundled.** The analyst runs in `deterministic` mode: every statement is computed from data and cites its evidence. A narrative summary needs an adapter registered in `app/services/ai_provider.py`.
- **Prices** are read from `$NN.NN` text and labelled USD. Distances are in miles.
- **Product matching** is token-based. A different brand whose title contains every requested word can still match.
- **Organic search results** carry no geography. An out-of-area business found only that way is recorded as `discovered` with "location not verified".
- **Price Intelligence results are not stored**, so they do not feed Trends, Market Pulse or the AI Analyst.
- **No authentication.** Run locally only, because the search endpoints spend SerpApi credits.

11. Database safety

The application database is `backend/data/market_radar.db`. It holds cached SerpApi responses (paid for) and usage history.

- Tests never touch it.
- The demo seeder refuses to write to it.
- The app only ever adds tables and columns; it never drops them.

Back the file up before experimenting.

12. Packaging for submission

Include the source, `docs/`, both `.env.example` files and this README. **Exclude:**

- `backend/.env` (contains your real API key)
- `backend/venv/`
- `frontend/node_modules/`
- `frontend/dist/`
- `backend/data/*.db`
- `_quarantine/`
- `__pycache__/`
- `.pytest_cache/`

The reviewer recreates the database and environments by following section 4.

13. Roadmap

- Register a real AI provider adapter for narrative summaries.
- PostgreSQL with managed migrations (Alembic).
- Multi-currency prices and unit conversion; store Price Intelligence results.
- Authentication and rate limiting before any non-local deployment.